|사물인터넷 실습|

Internet of Things and Laboratory

Android

강원대학교 컴퓨터공학과 권용훈

안드로이드 (1/3)

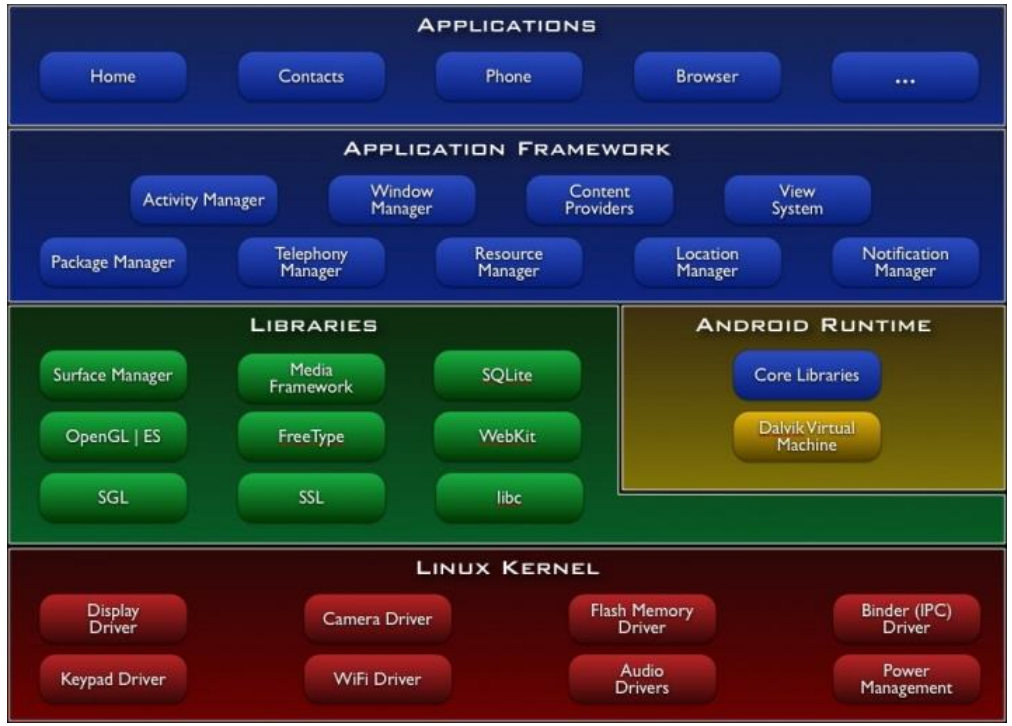
안드로이드(Android)

- OS, Middleware, Key Application을 포함하는 모바일 기기의 S/W 집합체
- 안드로이드 스튜디오 : 안드로이드 기기에 앱을 탑재하기 전에 PC에서 가상으로 앱을 제작하여 시험할 수 있는 프로그램
- 안드로이드 SDK : 안드로이드 플랫폼 상에서 자바 프로그래밍 언어를 사용하여 개발을 시작할 수 있도록 도구와 API를 제공
- 안드로이드의 특징
  - Application Framework
  - Virtual Machine
  - Integrated browser
  - Optimized graphics
  - SQLite
  - 디바이스 에뮬레이터, 디버깅 도구, 메모리 및 성능 프로파일링, Eclipse IDE 플러그인, etc.

안드로이드 (2/3)

안드로이드 구조

- 안드로이드 어플리케이션
  - Activity, Intent, Service, Content Provider 로 구성
  - 컴포넌트들의 목록을 AndroidManifest.xml 에 기록
  - xml: 선언된 컴포넌트의 기능과 요구사항을 기록하여 저장하는 파일
- Activity
  - 어플리케이션에서 하나의 화면을 지칭
  - Activity base class를 상속하여 구현한 하나의 클래스
  - 사용자에게 view와 event 응답을 제공하는 인터페이스



안드로이드 구조도

안드로이드 (3/3)

안드로이드 구조

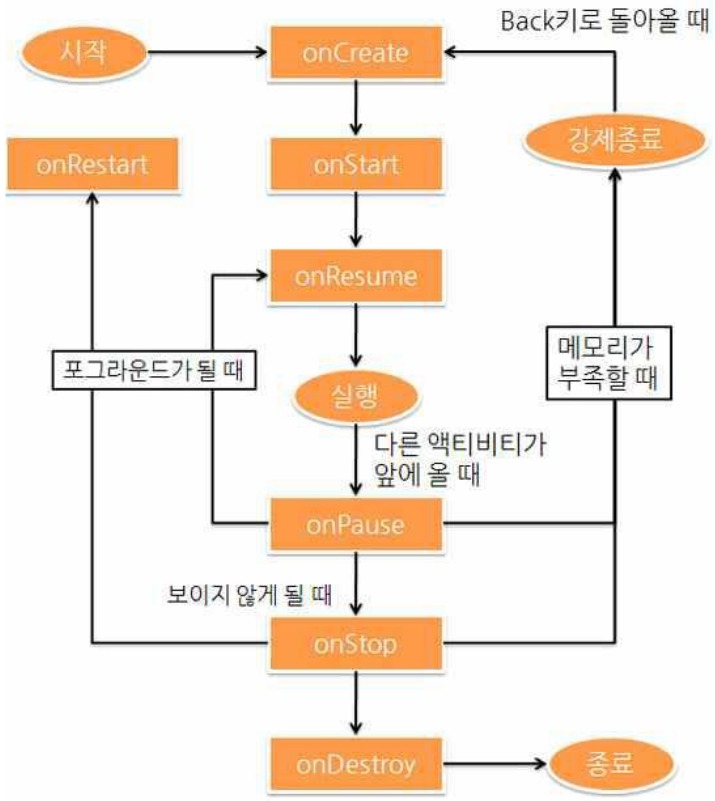
- Intent
  - Intent 클래스
    - 화면과 화면 사이를 이동할 때 사용하는 클래스
    - Intent가 어떤 작업에 대한 요청
    - 어플리케이션이 원하는 것에 대한 정보를 담고 있다.
  - IntentFilter 클래스
    - Activity(또는 Intent Receiver) 상의 Intent를 핸들링 할 수 있는지를 표현
    - Activity는 AndroidManifest.xml 파일에 기록된 IntentFilter를 생성
  - Intent Receiver
    - 작성한 어플리케이션 코드에서 핸드폰에 통화가 걸려오거나
      데이터 네트워크 접속이 활성화된 것과 같은 외부 이벤트를 처리하는 데 사용
    - NotificationManager를 이용하여 사용자에게 어떤 일이 발생했다는 것을 알려준다.
    - AndroidManifest.xml 파일에 등록되지만, Context.registerReceiver()를 이용하여 코드상에 등록 가능

- Service
  - UI와 상관없이 아주 오랫동안 존재하며 실행되는 코드
    예) 재생목록에서 노래를 재생하는 미디어 플레이어
- Content Provider
  - 자신의 데이터를 SQLite 데이터베이스 또는 다른 방법으로 파일에 저장할 수 있다.
  - 어플리케이션의 데이터가 다른 어플리케이션과 공유할 필요가 있을 때 유용
  - 다른 어플리케이션이 데이터를 저장하거나, 가져오는 작업을 할 수 있도록 한다.

어플리케이션 구조 (1/7)

Android Activity

- 안드로이드 디바이스에서 화면에 표시되는 UI 구성 요소의 집합
- 하나의 화면에는 하나의 Activity만 실행 가능
- 각 요소를 객체를 생성하여 원하는 동작을 구현 가능
- Java, Kotlin 으로 구현



Activity의 생명 주기

어플리케이션 구조 (2/7)

Android XML

- 사용자에게 보여지는 레이아웃을 작성하는 파일

Android Manifest

- 어플리케이션의 권한을 설정하는 파일
- Activity, library, version 에 대한 정보를 명시할 수 있다.

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.IoTPractice">

    <uses-permission android:name="android.permission.BLUETOOTH" />
    <uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
    <uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
```

Manifest 의 일부

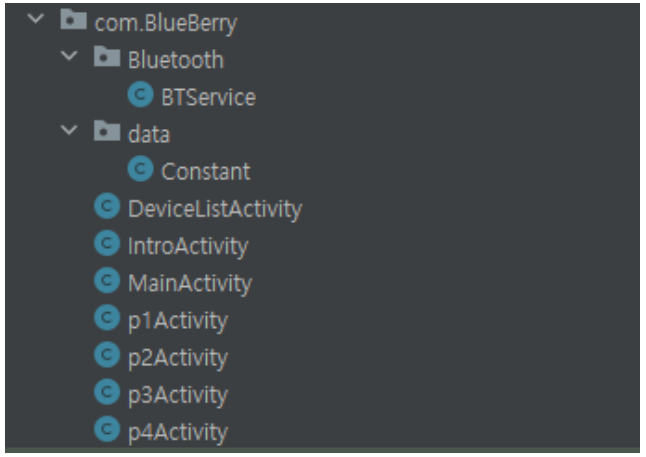
```xml
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@drawable/bg" >

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="@drawable/bg_title"
        android:gravity="center_vertical" >
        <!--
        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="@string/app_name"/>  -->

        <ImageView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:src="@drawable/new_title"
            android:contentDescription="임베디드시스템 블루투스"/>

        <Button
            android:id="@+id/btBTN"
            android:layout_width="50dp"
            android:layout_height="wrap_content"
            android:layout_marginRight="20dp"
            android:background="@drawable/selector_bt_btn"
            android:onClick="clickFunc"
            android:visibility="gone"/>
```

MainActivity의 XML

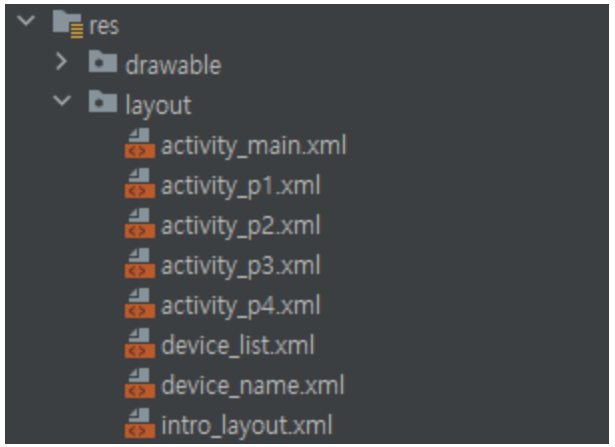
어플리케이션 구조 (3/7)

IoTPractice Application

- 아두이노에 연결된 센서 및 장치를 제어하기 위한 안드로이드 어플리케이션
- Activity와 Layout으로 구성
- IntroActivity, MainActivity 그리고 과제 수행을 위한 Activity
- 각 Activity을 화면상에 표현하는 xml 파일



Activity List



XML List

어플리케이션 구조 (4/7)

프로토콜(Protocol)

- 서로 다른 장비 사이에 데이터를 송수신하기 위한 규칙을 정하는 것
- 일반적으로 다음과 같은 규칙을 갖는다.

시작 문자	제어센서 이름	구분자	센서 제어값	종료 문자

- 실습용 예제는 다음과 같은 프로토콜을 채택

시작문자	구분자	종료문자
@	,(콤마)	#

[AI Solution Kit ==> App]

커맨드 (제어 센서 이름)	데이터 (센서 제어 값)	동작	예시
TMP	두 자리 숫자	온도 값 전달	@TMP,17#
HUM	두 자리 숫자	습도 값 전달	@HUM,45#
IMG	JPG이미지 데이터 마지막 3바이트 ###	이미지 데이터 전달	@IMG,..####

[App ===> AI Solution Kit]

커맨드 (제어 센서 이름)	데이터 (센서 제어 값)	동작	예시
RED	한 자리 숫자 0 – OFF 1 – ON	Red LED On/Off	@RED,0# @RED,1#
YED		Yellow LED On/Off	@YED,0# @YED,1#
GED		Green LED On/Off	@GED,0# @GED,1#
LED		LED On/Off	@LED,0# @LED,1#
SHT		카메라 촬영	@SHT#

어플리케이션 구조 (5/7)

MainActivity & activity_main.xml

- xml에서는 버튼, 토글 버튼, 텍스트 뷰 등의 요소를 추가하여 사용자 인터페이스를 구성

버튼 – activity_main.xml

```
<Button
    android:id="@+id/red_btn"
    android:layout_width="0dp"
    android:layout_weight="1"
    android:layout_height="wrap_content"
    android:text="@string/red_led"
    android:textStyle="bold"
    android:onClick="clickFunc"/>
```

- id는 해당 객체의 고유한 이름을 나타내며, onClick은 해당 버튼을 눌렀을 때의 동작을 나타낸다.
- width/weight/height 등의 요소는 객체의 모양과 위치를 결정하는 요소이다.

어플리케이션 구조 (6/7)

MainActivity & activity_main.xml

- xml에서 함수를 호출하면 해당 xml과 연결된 activity에서 해당 함수를 실행

clickFunc() – MainActivity

switch(v.getId()) {
  case R.id.*tg_btn_buzzer*:
  sendMessage((((ToggleButton) v).isChecked() == true) ? "@BUZ,1#" : "@BUZ,0#");
  break;
  case R.id.*red_btn*:
  ((Button) v).setTextColor((bRedOn == false) ? Color.*RED* : Color.*BLACK*);
  sendMessage((bRedOn == false) ? "@RED,1#" : "@RED,0#");
  bRedOn = !bRedOn;
  break;

- 눌려진 버튼의 id를 바탕으로 적절한 기능을 수행한다.

어플리케이션 구조 (7/7)

MainActivity & activity_main.xml

- initUI()는 xml에 있는 요소들을 MainActivity에서 초기화한다. findViewById() 함수를 통해 연결된다.

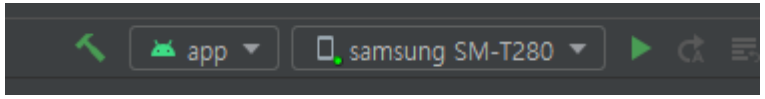
initUI() – MainActivity

private void initUI(){
mAppRes = this.getResources();
mBtBTN = (Button)findViewById(R.id.*btBTN*);
mBtBTN.setVisibility(View.*VISIBLE*);
mBtBTN.setBackgroundResource(R.drawable.*selector_bt_btn*);
mBtBTN.setSelected(false);
mTvTemp = findViewById(R.id.*tv_temp*);
mRedBtn = findViewById(R.id.*red_btn*);
mGreenBtn = findViewById(R.id.*green_btn*);
mBlueBtn = findViewById(R.id.*blue_btn*);
mBTService = new BTService(this, mHandler);
}

안드로이드 스튜디오 설정

안드로이드 - 갤럭시 탭 설정

- e-루리에서 'IoTPractice' 어플리케이션 소스 파일을 다운받아 안드로이드 스튜디오에서 실행한다.
- 갤럭시 탭을 케이블을 이용하여 PC에 연결한다.
- 갤럭시 탭 [설정] - [디바이스 정보] - [소프트웨어 정보] - [빌드 번호] 를 여러 번 클릭한다.
- [시스템] - [개발자 옵션] - [USB 디버깅] 이 활성화 되어있는지 확인한다.
- 연결 과정에 권한 승인 요청 메시지가 출력되면 승인한다.
- 안드로이드 스튜디오 오른쪽 상단에 연결 여부를 확인한다.



- 안드로이드 스튜디오 오른쪽 상단의 화살표를 눌러 빌드하여 설치한다.

블루투스 모듈 설정

블루투스 모듈 - 갤럭시 탭 설정

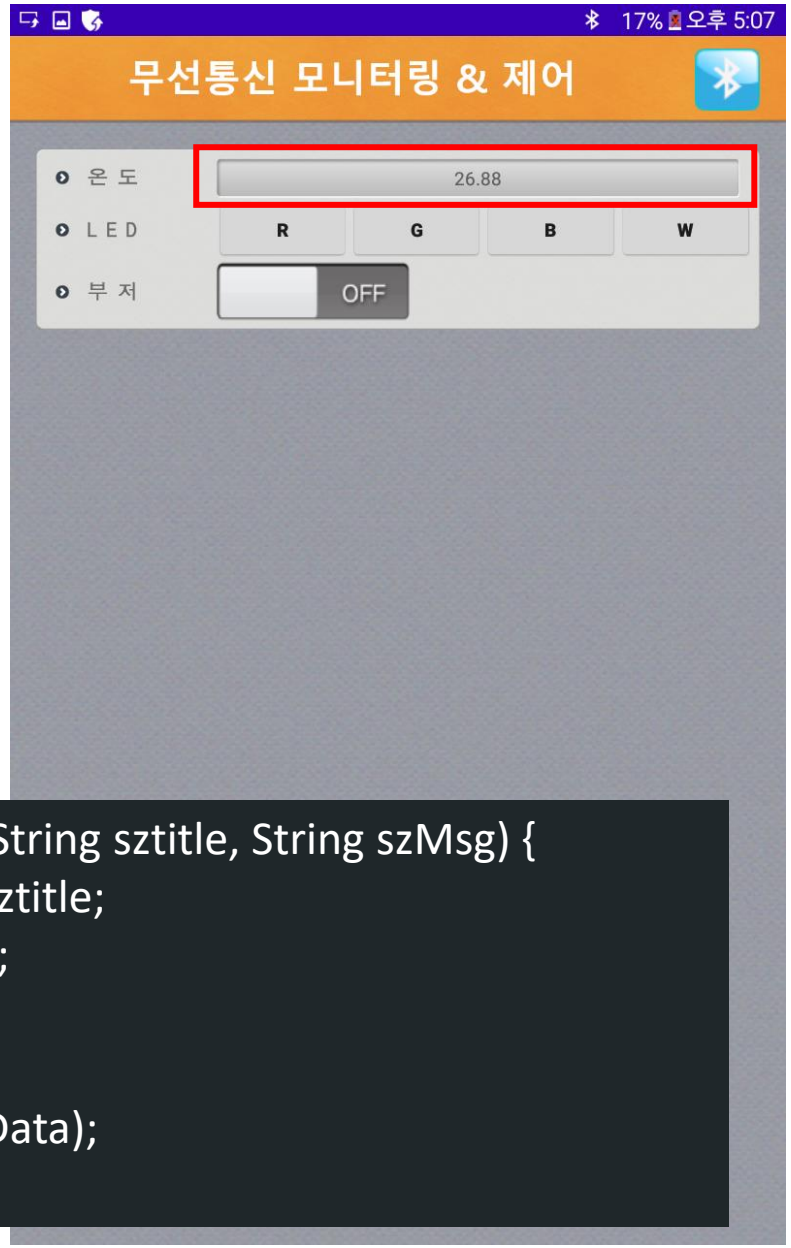
- 블루투스 모듈 중 Slave 모듈로 사용한 모듈을 이용
- AT 명령어를 이용하여 이름은 - IoT[XXXX] 로 설정
  - XXXX = 키트 번호 ex) 0001
- 블루투스 모듈과 페어링

블루투스 송신 예제

주기적으로 온도 센서의 데이터를 앱으로 전송

```
#include <SoftwareSerial.h>
#include <Thermistor.h>
#define RXD_PIN 2
#define TXD_PIN 3
Thermistor temp(A2);
unsigned long preMillis = 0;
void setup() {
  Serial.begin(9600);
  bt.begin(9600);
}
void loop() {
  if(millis() - preMillis >= 1000){
    sendStringData("@TMP,",
String(temp.getTemp() ) );
    preMillis = millis();
  }
}
```

```
void sendStringData(String sztitle, String szMsg) {
String szSendData = sztitle;
szSendData += szMsg;
szSendData +="#";
bt.print(szSendData);
Serial.println(szSendData);
}
```



블루투스 수신 예제

LED - R 버튼을 눌러 아두이노의 빨간색 LED 조작

```
#include <SoftwareSerial.h>
#define RXD_PIN 2
#define TXD_PIN 3
#define LED_RED_PIN 7
SoftwareSerial bt(RXD_PIN, TXD_PIN);
String rxData="";
void setup() {
  Serial.begin(9600);
  bt.begin(9600);
  pinMode(LED_RED_PIN, OUTPUT);
}
```

```
void loop() {
  if(bt.available()>0){
    rxData = bt.readString();
    if(rxData.startsWith("@") &&
rxData.endsWith("#"))
      controlData(rxData);
  }
}
void controlData(String rxData){
  Serial.print("RX <=== ");
  Serial.println(rxData);
  if( rxData.indexOf("RED,1") >=0 ){
    digitalWrite(LED_RED_PIN, HIGH);
  }else if( rxData.indexOf("RED,0") >=0 ){
   digitalWrite(LED_RED_PIN, LOW);
  }
}
```

고생하셨습니다

Thank you!